DriftConditions Design Doc

DriftConditions is an online streaming audio source that captures the chaos and serendipity of late-night radio tuning in an uncanny audio stream generated on the fly by code. Overlapping fragmented stories, ambient sounds, and mysterious crosstalk weave a vivid sonic tapestry that draws listeners into an immersive and unpredictable listening experience. Inspired by the unpredictability of real-world radio interference, DriftConditions explores the boundaries between intention and happenstance, inviting listeners to eavesdrop on a hidden world of voices and atmospheres unconstrained by traditional narrative structures. With each new listening session, DriftConditions offers a fresh journey through its evocative auditory landscape.

How It Works

Front End

Audio stream out will allow users to connect via the internet and listen. Controls are minimal.

Editor Back End

Front end users can:

- connect to the server to manage audio sources:
    - browse - view/listen to audio with selection filters
    - add - upload new audio
    - edit - normalize, adjust volume, trim ends, etc
    - clasify - determine the type to be used in recipes
    - approve/disapprove - enable or disable for play
    - mark for review - pass upward to higher role
    - trash - disapproves and marks for deletion
    - delete - permanently delete audio
- create or modify recipes
- do different functions according to different roles
- access admin interface for managing permissions and roles

Audio Back End

The audio server:

- accesses the database tables and audio sources
- only has access to approved audio
- chooses random recipes (maybe tracery grammar expansions?)
- chooses random audio sources that meet recipe requirements
- manages multiple tracks including
    - assigns audio to track slots according to recipes
    - manages volume of different tracks according to recipes

Database

Tables we'll need:

- Users
- Roles/Permissions
- Audio
- Recipes
- Audit

Schema

Users Table

- user_id (Primary Key): A unique identifier for each user.
- username: A unique name chosen by the user for login or identification.
- email: User's email address.
- password: Hashed password for secure storage.
- role_id (Foreign Key): Links to the Roles/Permissions table to assign user roles.

Roles Table

- role_id (Primary Key): A unique identifier for each role.
- role_name: The name of the role (e.g., admin, editor, guest).
- permissions: A list or JSON object detailing the specific actions that the role is allowed to perform. This could be structured according to your application's

needs.

Audio Table

- audio_id (Primary Key): A unique identifier for each audio file.
- title: Name or title of the audio clip.
- filename: name and path of file
- uploader_id (Foreign Key): user_id of the user who uploaded the file.
- upload_date: The date and time when the audio was uploaded.
- editor_id (Foreign Key): user_id of the user who last edited the audio file.
- edit_date: The date and time when the audio was edited.
- duration: Length of the audio file.
- file_type: The format of the audio file (mp3, wav, etc.).
- status: Current status (approved, disapproved, under review, trashed).
- classification: Type or category of the audio to be used in recipes.
- tags: Arbitrary classification tags
- comments: Notes of uploader, editor, or admin

Recipes Table

- recipe_id (Primary Key): A unique identifier for each recipe.
- recipe_name: The name of the recipe.
- creator_id (Foreign Key): user_id of the user who created or last modified the recipe.
- create_date: The date and time when the recipe was created.
- updater_id (Foreign Key): user_id of the user who last edited the recipe.
- update_date: The date and time when the recipe was last updated.
- description: A brief description of what the recipe does.
- recipe_data: A JSON object or similar structure that contains the specifics of how audio should be mixed, including volume levels, track assignments, and any other relevant parameters.
- comments: Notes of creator, updater, or admin

Audit Table

- audit_id (Primary Key): Unique identifier for each audit record.
- table_name: The name of the table where the change occurred.

- record_id: The primary key of the record that was changed, as a string or appropriate type.
- action_type: The type of action performed (INSERT, UPDATE, DELETE).
- action_by: user_id of the user who made the change.
- action_date: The timestamp of when the change was made.

Entity-Relationship Diagram

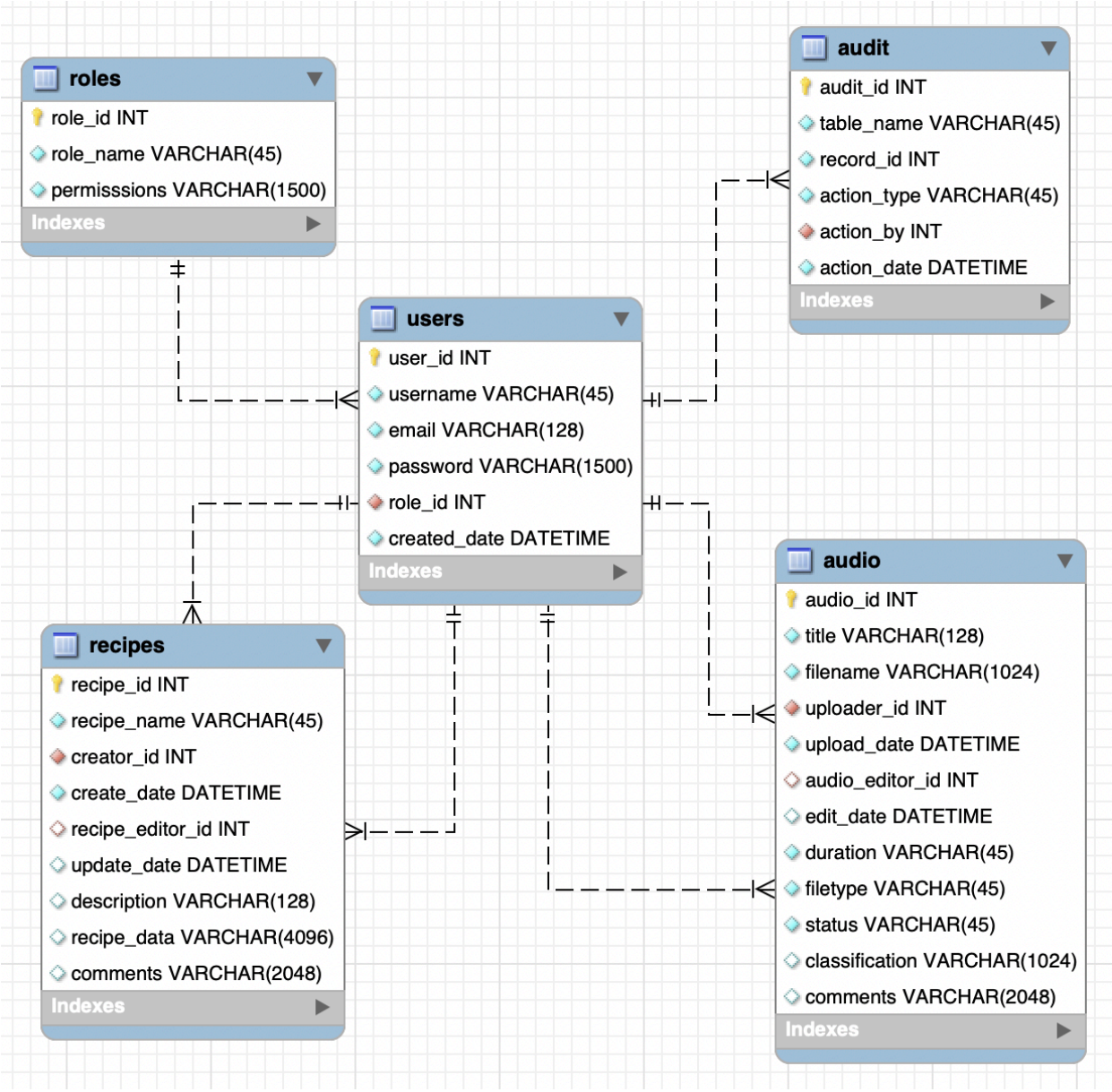


Table Creation via SQL

```
CREATE TABLE `roles` (
```

```
  `role_id` int NOT NULL AUTO_INCREMENT,
  `role_name` varchar(255) NOT NULL,
  `permissions` json DEFAULT NULL,
  PRIMARY KEY (`role_id`),
  UNIQUE KEY `role_id_UNIQUE` (`role_id`),
  UNIQUE KEY `role_name_UNIQUE` (`role_name`)
) ENGINE=InnoDB AUTO_INCREMENT=5 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

INSERT INTO `interference`.`roles` (`role_name`) VALUES ('admin');
INSERT INTO `interference`.`roles` (`role_name`) VALUES ('mod');
INSERT INTO `interference`.`roles` (`role_name`) VALUES ('editor');
INSERT INTO `interference`.`roles` (`role_name`) VALUES ('contributor');
INSERT INTO `interference`.`roles` (`role_name`) VALUES ('user');

CREATE TABLE `users` (
  `user_id` int NOT NULL AUTO_INCREMENT,
  `username` varchar(255) NOT NULL,
  `email` varchar(320) NOT NULL COMMENT 'According to email standards (RFC 5321 and RFC 5322),
the maximum length of an email address is 320 characters',
  `password` varchar(1024) NOT NULL,
  `role_id` int NOT NULL DEFAULT '4',
  PRIMARY KEY (`user_id`),
  UNIQUE KEY `username` (`username`),
  UNIQUE KEY `user_id_UNIQUE` (`user_id`),
  KEY `fk_users_roles_idx` (`role_id`),
  CONSTRAINT `fk_users_roles` FOREIGN KEY (`role_id`) REFERENCES `roles` (`role_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

CREATE TABLE `audio` (
  `audio_id` int NOT NULL AUTO_INCREMENT,
  `title` varchar(255) NOT NULL,
  `filename` varchar(255) NOT NULL,
  `uploader_id` int NOT NULL,
  `upload_date` datetime NOT NULL,
  `editor_id` int DEFAULT NULL,
  `edit_date` datetime DEFAULT NULL,
  `duration` int NOT NULL,
  `file_type` varchar(50) NOT NULL,
  `status` varchar(32) NOT NULL DEFAULT 'REVIEW' COMMENT 'approved, disapproved, under review,
trashed',
  `classification` varchar(255) DEFAULT NULL,
  `tags` json DEFAULT NULL,
  `comments` varchar(2048) DEFAULT NULL,
  PRIMARY KEY (`audio_id`),
  UNIQUE KEY `audio_id_UNIQUE` (`audio_id`),
  KEY `fk_audio_users1_idx` (`uploader_id`),
  KEY `fk_audio_users2_idx` (`editor_id`),
  CONSTRAINT `fk_audio_users1` FOREIGN KEY (`uploader_id`) REFERENCES `users` (`user_id`),
  CONSTRAINT `fk_audio_users2` FOREIGN KEY (`editor_id`) REFERENCES `users` (`user_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

CREATE TABLE `recipes` (
  `recipe_id` int NOT NULL AUTO_INCREMENT,
```

```
  `recipe_name` varchar(255) NOT NULL,
  `creator_id` int NOT NULL,
  `create_date` datetime NOT NULL,
  `updater_id` int DEFAULT NULL,
  `update_date` datetime DEFAULT NULL,
  `description` text NOT NULL,
  `recipe_data` json DEFAULT NULL,
  `comments` varchar(2048) DEFAULT NULL,
  PRIMARY KEY (`recipe_id`),
  UNIQUE KEY `recipe_id_UNIQUE` (`recipe_id`),
  UNIQUE KEY `recipe_name_UNIQUE` (`recipe_name`),
  KEY `fk_recipes_users1_idx` (`creator_id`),
  KEY `fk_recipes_users2_idx` (`updater_id`),
  CONSTRAINT `fk_recipes_users1` FOREIGN KEY (`creator_id`) REFERENCES `users` (`user_id`),
  CONSTRAINT `fk_recipes_users2` FOREIGN KEY (`updater_id`) REFERENCES `users` (`user_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

CREATE TABLE `audit` (
  `audit_id` int NOT NULL AUTO_INCREMENT,
  `table_name` varchar(255) NOT NULL,
  `record_id` varchar(255) NOT NULL,
  `action_type` varchar(32) NOT NULL COMMENT '''INSERT'', ''UPDATE'', ''DELETE''',
  `action_by` int NOT NULL,
  `action_date` datetime NOT NULL,
  PRIMARY KEY (`audit_id`),
  UNIQUE KEY `audit_id_UNIQUE` (`audit_id`),
  KEY `fk_audit_users_idx` (`action_by`),
  CONSTRAINT `fk_audit_users` FOREIGN KEY (`action_by`) REFERENCES `users` (`user_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

Necessary Pages

Public Facing

✅ Stream Page

- Accessible to any registered user
- Access to current outbound stream

Users

✅ Signin Page

✅ Signup Page

✅ Profile Page

- Access controlled by roles (users, contributors, editors, mods, and admins)
- Displays username, firstname, lastname, email, added date, role
- Mutable attributes (username, firstname, lastname) can be modified by user
- Protected attributes (role) access controlled by roles (mods and admins)
- Lists contributor's uploaded audio elements with links to edit each (if any)
- Lists contributor's created recipes with links to edit each (if any)
- Option to request upgraded role to contribute audio
- Comments field shows to mods and admins (controlled by roles)
- Element gives list of users requesting upgraded role (controlled by roles)
- Element gives list of newest audio to review (controlled by roles)

✅ User List

- Access controlled by roles (mods and admins)
- Links to profiles
- Filterable by role or date
- Sortable by number of audio and recipes
- Displays username, firstname, lastname, added date, numbers of uploaded audio, edited audio, created recipes, edited recipes

Roles

✅ Role Page

- Accessible to admins (always admins)
- Links to user list filtered by role
- Displays role name, number of users of each
- Lists accessible pages for each role (modifiable by admins)

Audio

✅ Audio Display Page

- Access controlled by roles (contributors, editors, mods, and admins)
- Links to audio edit page for this audio (controlled by roles)
- Links to audio list page

- Displays title, filename, uploader, upload date, editor, edit date, duration, file type, status, classification, tags, comments
- Displays wave form allowing listen

✅ Audio Edit Page

- Access controlled by roles (editors, mods, and admins)
- Accessible to contributors for their audio uploaded
- Links to audio display page for this audio
- Links to audio list page
- Displays title, filename, uploader, upload date, editor, edit date, duration, file type, status, classification, tags, comments
- Displays wave form allowing listen
- Contributors can modify: title, comments
- Admins, mods, and editors can modify: titles, status, classification, tags, comments
- Edit options: cut, copy, paste, apply effects
- Option to submit to database
- Option to download to local file
- Option to replace with upload
- Option to cancel edit
- Sets editlock if changes are made
- Releases editlock if submitted or canceled

✅ Audio List Page

- Access controlled by roles (editors, mods, and admins)
- Possible filter include by uploader, editor, date, classification, tags, or status (default is no filter)
- Possible sortable by creation date or edit date (default)
- Displays: title, uploader, upload date, editor, edit date, duration, file type, status, classification, tags (abreviated for tabular display)
- Links to view, edit, and trash are on line following audio data (controlled by roles)
- Pagination is provided at top and bottom. Default is 20 records per page.

Recipes

Recipe Display Page

- Access controlled by roles (mods, and admins)
- Links to recipe edit page for this recipe (controlled by roles)
- Links to recipe list page
- Displays recipe name, creator, create date, updater, update date, description, status, classification, tags, comments
- Graphically display recipe

Recipe Edit Page

- Access controlled by roles editors, mods, and admins)
- Accessible to users for their recipe uploaded
- Links to recipe display page for this recipe
- Links to recipe list page
- Displays name, creator, create date, updater, update date, description, status, classification, tags, comments
- Option to create test audio (download or stream)
- Graphically display recipe
- Modify: recipe name, creator, create date, updater, update date, description, status, classification, tags, comments
- Edit options: cut, copy, paste
- Option to submit to database
- Option to cancel edit

Recipe List Page

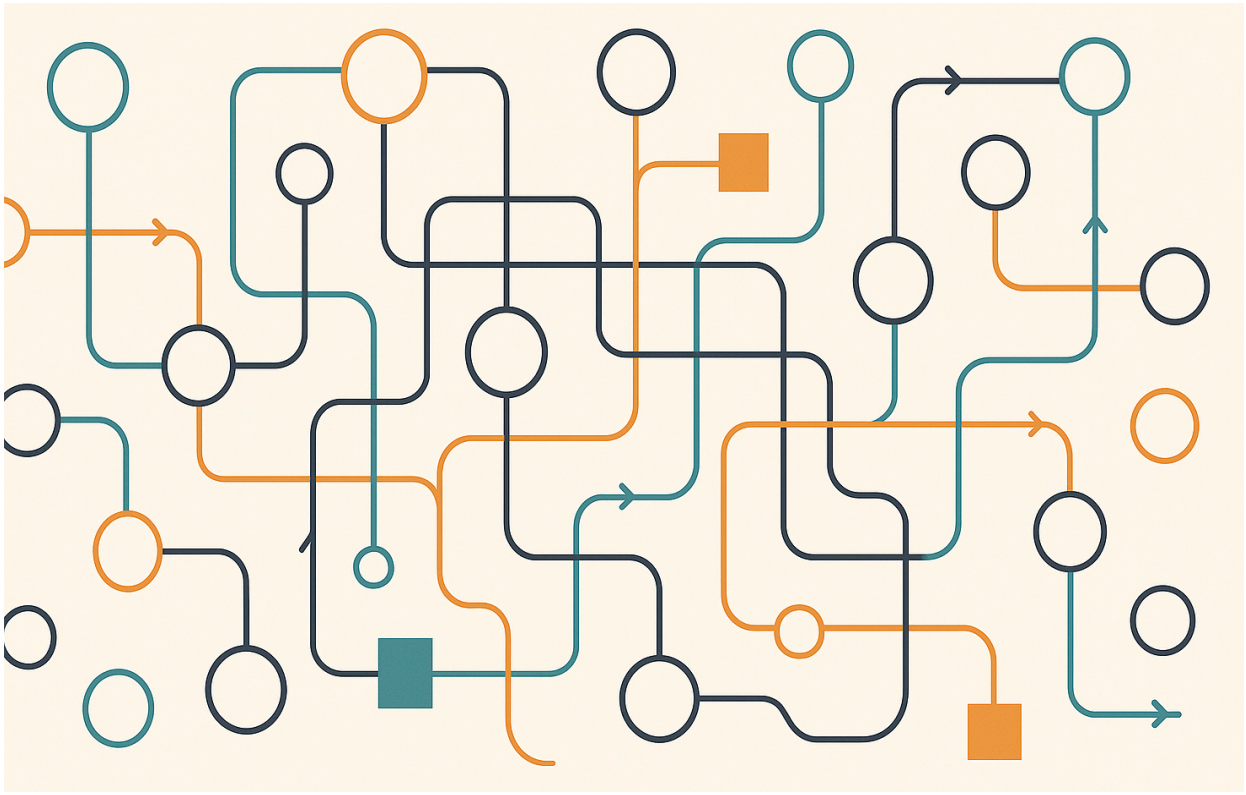
- Access controlled by roles (mods, and admins)
- Filter by creator, create date, updater, update date
- Sortable by creation date or edit date
- Displays: title, uploader, upload date, editor, edit date, duration, file type, status, classification, tags (abreviated for tabular display)
- Links to recipe display page or recipe edit page (controlled by roles)

Audit

Audit List Page

- Accessible to admins (always)
- Filter by table, record id, title of record, action type, action by, action date
- Sortable by action date
- Displays: table, record id, title of record, action type, action by, action date
- Links to item in user, audio, or recipe page

Authentication and Authorization



Overview

The authentication and authorization system uses JSON Web Tokens (JWT) to manage user sessions. The server enforces role-based access control using permissions encoded in the token, while the React app adapts its UI dynamically based on the user's role.

UserID Management

- Each user has a unique `userID` stored in the database.
- User records include `userID`, `username`, `email`, `roleName`, and `permissions`.
- The server decodes the `userID` and `permissions` from the session token to validate access and fetch user-specific data.

Session Token Handling

Token Assignment

- Upon sign-in, the server generates a session token containing:
  - `userID`: Identifies the user.
  - `username`: Personalizes the UI.
  - `permissions`: Determines accessible actions and pages.
- The token is stored in an HTTP-only cookie with:
  - `path: '/'`
  - `sameSite: 'None'` or `'Strict'`
  - `secure: true` (in HTTPS environments).
- The token expiration is determined by the constant `tokenExpires`.

Token Validation

- The token is validated on every request except for the following pages:
  - `homepage`
  - `Error`
  - `NotAuth`
  - `Signin`
  - `Signup`
- If invalid or missing, the server denies access and redirects the user.

Token Refresh

- The server issues a new token when:
  - The current token is near expiration (less than one hour remaining).

    - The user's permissions have changed since the token was issued.

Authorization

Authorization determines what resources and actions users can access based on their role and permissions.

Key Concepts

- **Permissions**: Each role has a set of permissions (e.g., `audioList`, `recipeCreate`) defined in the database.
- **Role-Based Access Control**: Roles like `admin` or `contributor` determine the permissions assigned to a user.
- **Token-Based Enforcement**: Permissions are embedded in the JWT token during login and validated on the server for protected pages or APIs.

Server-Side Authorization

1. During login, the server:
    - Fetches the user's role and associated permissions from the database.
    - Encodes the permissions in the JWT token.
2. For protected pages and APIs:
    - The server decodes the session token to retrieve the `permissions` array.
    - Compares the requested page or action (`context`) with the permissions list.
    - If unauthorized, the server responds with a 403 (Access Denied) status.

Frontend Authorization

1. **Dynamic Menuing**:
    - The React app uses the permissions array stored in Redux to determine visible menu items.
    - Example:
        - Admin users see links like "Manage Users" or "Role Management."

        - Contributors see options for uploading and editing content.
        - Unauthenticated users only see public links like `Signin` or `Signup`.
2. **Protected Pages**:
    - The `useAuthCheckAndNavigate` hook redirects unauthorized users to `/notauth` if they lack the required permissions for a page.

Signup and Signin

Signup

- Users register via the `/signup` endpoint.
- The server:
    - Validates input.
    - Hashes passwords with `bcrypt`.
    - Inserts user data into the database.
    - Redirects successful signups to the `Signin` page.
- Duplicate usernames or emails return a 409 error.

Signin

- Users log in via the `/signin` endpoint.
- The server:
    - Verifies credentials against the database.
    - Issues a JWT token on success.
    - Responds with a 418 error for invalid credentials.

React App Integration

Dynamic UI Updates

- Menu visibility and homepage content adapt based on the user's `permissions`, which are stored in Redux state along with user info.
- Examples:
    - Admin users see links to admin pages.

    - Contributors access content creation tools.
    - Unauthenticated users see only public links.

State Management

- Redux stores non-sensitive user data like `username` for UI consistency.
- Sensitive data (e.g., `permissions`) is fetched securely during authentication checks.

Additional Considerations

Logout

- The `/logout` endpoint invalidates the session token by expiring the cookie.
- The server confirms the logout to the client.

Rate Limiting

- Authentication endpoints (`/signup`, `/signin`) should enforce rate limits to prevent brute-force attacks.
- *Future note:* Implement fine-grained rate limiting by IP or client behavior.

How Authentication and Authorization Work

When a user visits the app, the React frontend triggers an initial authentication check. The server validates the session token and decodes user details (`userID`, `username`, `permissions`). These details inform the client-side app about access levels.

During signup, the server hashes user passwords securely and creates a database record. After registration, the user is redirected to the login page.

For sign-in, the server validates credentials and issues a JWT token. This token encodes the user's identity and permissions and is stored in a secure cookie. The user info is passed to the client and stored in Redux state.

The React app uses the `permissions` array to generate menus and tailor content.

Admins access privileged features, contributors manage their work, and public users see only basic options.

For logout, the server expires the session token, and the React app resets the state. Future enhancements include rate limiting, stricter token refresh policies, and improved error handling.

Authentication and Authorization Testing

Here's a strategic list of tests to ensure the **authentication and authorization (auth/auth)** process is functioning correctly. The tests are categorized to cover key scenarios, avoiding redundant tests while ensuring thorough coverage.

General Tests (Unauthenticated User)

- Unauthenticated user sees `noauth` menu. local | server
- Unauthenticated user can visit `homepage`. local | server
- Unauthenticated user can visit `signin` page. local | server
- Unauthenticated user can visit `signup` page. local | server
- Unauthenticated user can visit `error` page. local | server
- Unauthenticated user can NOT visit `profile` page. local | server
- Unauthenticated user can NOT visit `audio list` page. local | server

User Role Tests

- User with role `user` is able to login. local | server
- User with role `user` is redirected to `profile` page. local | server
- User with role `user` sees `user` menu. local | server
- User with role `user` can visit `homepage`. local | server
- User with role `user` can visit `signin` page. local | server
- User with role `user` can visit `signup` page. local | server
- User with role `user` can visit `error` page. local | server
- User with role `user` can visit `profile` page. local | server
- User with role `user` can NOT visit `audio list` page. local | server

Admin Role Tests

- User with role `admin` is able to login. local | server
- User with role `admin` is redirected to `profile` page. local | server
- User with role `admin` sees `admin` menu. local | server
- User with role `admin` can visit `homepage`. local | server
- User with role `admin` can visit `signin` page. local | server
- User with role `admin` can visit `signup` page. local | server
- User with role `admin` can visit `error` page. local | server
- User with role `admin` can visit `profile` page. local | server
- User with role `admin` can visit `audio list` page. local | server

Session and Token Tests

- Valid session token grants access to a protected page without re-login. local | server
- Expired session token redirects the user to the `signin` page. local | server
- Tampered session token redirects the user to the `signin` page. local | server
- Session token nearing expiration triggers automatic token refresh. local | server

Logout Tests

- Logging out invalidates the session token. local | server
- Logged-out user sees `noauth` menu. local | server
- Logged-out user is redirected to `signin` when visiting a protected page. local | server

Miscellaneous Tests

- Permissions update for a logged-in user triggers token refresh. local | server
- Unauthenticated user attempting to access a restricted API endpoint receives a 403 response. local | server
- Authenticated user with valid permissions can access a restricted API endpoint. local | server
- Authenticated user with insufficient permissions attempting to access a restricted API endpoint receives a 403 response. local | server

Test Notes

- **Focus on Strategic Pages**: Test `homepage`, `signin`, `signup`, `profile`, and one example of a restricted page like `audio list`.
- **Minimal Repetition**: Avoid testing all pages for all roles; focus on representative scenarios for each role (`user`, `admin`).
- **API Tests**: Cover backend responses for unauthorized or authenticated requests, especially for session validation (`/check`) and role permissions updates.

Audio File Naming and Storage

Filenames

Allowing uploaded filenames to keep their original names when stored presents a mix of advantages and disadvantages, impacting file management, user experience, and system integrity.

Advantages

- User Familiarity: Enhances user experience with recognizable file names.
- Simplicity: Simplifies the upload process, reducing backend complexity.
- Direct Reference: Makes links to files more meaningful and memorable.

Disadvantages

- Security Risks: Potential for malicious code in original filenames.
- Filename Collisions: High risk of overwriting files with the same name.
- Special Characters and Encoding Issues: Can cause compatibility problems.
- Unintended Disclosure: Filenames might reveal sensitive information.

Alternatives

- Generating Unique Identifiers: Ensures uniqueness and improves security.
- Timestamps and User ID Combination: Differentiates files while maintaining references.
- Content Hashing: Guarantees uniqueness and supports deduplication.

- Directory Structures: Reduces collision risk and improves file management without changing filenames.
- Sanitization and Normalization: Removes special characters and standardizes filenames while keeping them recognizable.

Storage

Managing a large number of files in a single directory can indeed become problematic due to issues like filename collisions and performance degradation when accessing the files. Several strategies have been adopted by content management systems (CMS) and other applications to subdivide upload directories efficiently:

Date and Time-Based Directories:

- Structure: Files are stored in directories based on the date and time they were uploaded. This could be as granular as year/month/day/hour or simply year/month.
- Example CMS Use: WordPress uses this method by default, creating a new directory for each year and month (e.g., /uploads/2024/02/).
- Advantages: Simplifies finding files by date, evenly spreads out files over time, and prevents directories from getting too large.
- Considerations: May not prevent directory bloat if many files are uploaded in a short period.

Notes on Creating Soundscapes

Taking with Jacin abotu Clatterbox

- Soundscrapes could be found records, or combined audio sources, mixed between them by "riding the flow" of the audio
- Sources could include traditional music, electronic music, music, instructional records, childrens records
- Over this maybe there was narrative, with the soundscrape potted up or down at key moments
- The audio sources might be slowed down or sped up or reversed

- Over all of this sound effects could be added
- Sound effects might be noisy toys, voice modification, or tape payer manipulation

Channels for DriftConditions

- channels / classification
    1. Ambient - longer ambient recordings
    2. Background - longer audio or music
    3. Foreground - longer audio or music, e.g. narrative
    4. Interjections - medium length recordings
    5. Effects - short recorded efffects

Recipes

- How to represent the recipes in data?
- How to represent the recipes visually??
- Each channel should be able to have multiple layers

Audio

- Audio would have to be normalized to the average without clipping
- Add copyright clearance

Uploading Audio Routes

- the upload dir is "../upload" at the root of the project
- all upload will be upload/year/monthnum/filename, obv these dirs may have to be created
- filename will be the title converted to lowercase, all punctuation removed, and spaces replaces with dashes. Extensions will be preserved but changed to lowercase. ex: eavesdropping-in-a-cafe.mp3
- the filename added to the database will include the path minus the "../upload" root
- uploader_id will be the userID of the logged in user
- duration will have to be calculated. Not sure how to do that.
- file_type will be the lowercase extension
- for tags, we will go through them (separated by commas) and normalized by

converting to lowercase, removing special characters, trimming leading and trailing whitespace, and converting spaces to dashes. Also ensure that there are no duplicate tags.
- title, comments, and copyright_cert remain unchanged, though naturally we project against db injection

Generating Homepage Text

- DriftConditions is an online streaming audio source that captures the chaos and serendipity of late-night radio tuning in an uncanny audio stream generated on the fly by code.
- Overlapping fragmented stories, ambient sounds, and mysterious crosstalk weave a vivid sonic tapestry that draws listeners into an immersive and unpredictable listening experience.
- Inspired by the unpredictability of real-world radio interference, DriftConditions explores the boundaries between intention and happenstance, inviting listeners to eavesdrop on a hidden world of voices and atmospheres unconstrained by traditional narrative structures.
- With each new listening session, DriftConditions offers a fresh journey through its evocative auditory landscape.
- DriftConditions whispers its tales as secrets shared under the cloak of night, an intimate exchange between the listener and the vast, unseen world.
- It's told not with the clarity of daylight but with the mystery of shadows, where sounds blend and tales intertwine like conversations on a long, late-night journey with a close companion.
- Stories come through in fragmented pieces, an overheard conversation, a distorted broadcast, a larger narrative.
- It invites you to lean closer, to become part of an unfolding mystery, the warmth of voice and the chill of the unknown, all wrapped in the serendipity of an endless night drive.
- DriftConditions blends the unpredictability of late-night radio with the creativity of code-generated audio, crafting an immersive stream of stories, sounds, and crosstalk.
- This online audio experience weaves a tapestry of serendipitous encounters, drawing listeners into a world where intention and chance converge.

- Each session offers a unique journey through its evocative soundscape, inviting an eavesdrop into a realm beyond traditional narratives.
- DriftConditions is a online streaming audio source that captures the chaos and serendipity of late-night radio tuning in an uncanny audio stream generated on the fly by code.
- Overlapping fragmented stories, ambient sounds, and mysterious crosstalk weave a vivid sonic tapestry that draws listeners into an immersive and unpredictable listening experience.
- Inspired by the unpredictability of real-world radio interference, DriftConditions explores the boundaries between intention and happenstance, inviting listeners to eavesdrop on a hidden world of voices and atmospheres unconstrained by traditional narrative structures.
- With each new listening session, DriftConditions offers a fresh journey through its evocative auditory landscape.

FFMPEG ComplexFilter

```
/*
  Scenario 1: Single Track with One Clip
     [
     {
       filter: 'volume',
       options: 'volume=1.0',  // 100% volume
       inputs: '0',  // Assuming '0' refers to the first (and only)
input track
       outputs: 'final_output'
     }
   ]


 Scenario 2: Two Tracks with Independent Volume
   [
     {
       filter: 'volume',
       options: 'volume=0.5',  // 50% volume for track 0
```

```
      inputs: '0',  // Assuming '0' refers to the first input track
      outputs: 'track0_output'
    },
    {
      filter: 'volume',
      options: 'volume=1.0',  // 100% volume (no change) for track 1
      inputs: '1',  // Assuming '1' refers to the second input track
      outputs: 'track1_output'
    },
    {
      filter: 'amix',
      options: { inputs: 2 },  // Mix both tracks
      inputs: ['track0_output', 'track1_output'],
      outputs: 'final_output'
    }
  ]
```

Scenario 3: One Track, Sequential Clips

```
  [
    {
      filter: 'concat', // Concatenate the two audio clips into a
single track
      options: { n: 2, v: 0, a: 1 }, // 'n' is the number of audio
files, 'v' for video (none), 'a' for audio streams
      inputs: ['0', '1'], // Assuming '0' and '1' refer to the first
and second input tracks
      outputs: 'concatenated_output'
    },
    {
      filter: 'volume', // Set volume for the concatenated track
      options: 'volume=1.0', // 100% volume
      inputs: 'concatenated_output', // Use the output from the
concat filter
      outputs: 'final_output'
    }
  ]
```

Scenario 4: Two Tracks, Silence and Clips

```
   [
     {
       filter: 'anullsrc', // Generate silence
       options: { r: 44100, cl: 'stereo', d: 10 }, // Configure
duration and sample rate
       outputs: 'silence1'
     },
     {
       filter: 'volume', // Adjust volume for file1.mp3
       options: 'volume=1.0',
       inputs: '1', // Assuming '1' refers to file1.mp3
       outputs: 'file1_output'
     },
     {
       filter: 'anullsrc', // Generate silence
       options: { r: 44100, cl: 'stereo', d: 10 }, // Configure
duration and sample rate
       outputs: 'silence2'
     },
     {
       filter: 'concat', // Concatenate silences and file1.mp3 for
track 0
       options: { n: 3, v: 0, a: 1 }, // Three audio streams, no video
       inputs: ['silence1', 'file1_output', 'silence2'],
       outputs: 'track0_final'
     },
     {
       filter: 'volume', // Adjust volume for file2.mp3
       options: 'volume=0.5',
       inputs: '2', // Assuming '2' refers to file2.mp3
       outputs: 'track1_final'
     },
     {
       filter: 'amix', // Mix the two final tracks together
       options: { inputs: 2 }, // Mix two audio inputs
       inputs: ['track0_final', 'track1_final'],
       outputs: 'final_output'
     }
   ]
 */
```

Academic Issues Arise

DriftConditions, as a project, touches on several academic and artistic issues:

1. **Authorship and Creativity**: The project raises questions about authorship in art, especially with its use of procedural generation and user-contributed content. Who is the author of the art—the algorithm, the users contributing content, or the creators of the system?
2. **Generative Art and AI**: The use of algorithms to generate unique, non-repeating audio content brings up discussions about the role of AI and generative systems in contemporary art. It challenges traditional notions of creativity and artistic expression.
3. **Interactivity and Participation**: By allowing users to contribute audio and modify recipes, the project engages with ideas about interactivity in art. It explores how audience participation can influence the final artistic product, blurring the lines between creator and consumer.
4. **Sound Art and Experimental Music**: The project fits within the realms of sound art and experimental music, pushing boundaries in how sound is used as an art form. It invites discussions about the aesthetic value of ambient noise, non-linear narratives, and soundscapes in art.
5. **Ephemerality and Temporality**: The transient nature of the audio experiences, which are constantly changing and never repeated, raises issues around the ephemerality of digital art. It prompts reflection on how temporal aspects of art can affect perception and meaning.
6. **Cultural Sampling and Remix Culture**: DriftConditions might be viewed in the context of remix culture, where existing cultural artifacts are reinterpreted and repurposed. This brings up ethical and legal discussions about cultural sampling, appropriation, and the value of derivative works.

These issues are central to contemporary debates in digital art, sound studies, and cultural theory, making DriftConditions a rich subject for academic exploration.

John Cage

- Themes: Chance operations, questioning authorship

- Connection: DriftConditions' use of procedural generation and randomness in soundscapes
- Example: Cage's "4'33" challenges traditional artistic boundaries, similar to DriftConditions' dynamic audio content

Brian Eno

- Themes: Ambient music, generative compositions
- Connection: Continuous, immersive audio environments
- Example: Eno's "Music for Airports" and DriftConditions both create evolving auditory experiences that change with each listening session

Nam June Paik

- Themes: Media art, interactive installations
- Connection: Blend of human creativity and technological innovation
- Example: Paik's interactive works parallel DriftConditions' algorithmic curation and user-submitted audio clips

Christian Marclay

- Themes: Collage, transformation of recordings
- Connection: New auditory contexts through layered narratives
- Example: Marclay's use of existing recordings to create new works reflects DriftConditions' layering of audio elements

Ryoji Ikeda

- Themes: Data aesthetics, digital information
- Connection: Use of coherent noise techniques to simulate radio interference
- Example: Ikeda's exploration of digital sound aligns with DriftConditions' dynamic and unpredictable listening experiences

Holly Herndon

- Themes: AI, machine learning in music production
- Connection: Exploration of human-technology collaboration
- Example: Herndon's work with AI in music parallels DriftConditions' use of

algorithms to shape the audio landscape

These influences frame DriftConditions within contemporary art discussions on digital interactivity, procedural creativity, and the evolving relationship between technology and artistic expression.

Promotional

DriftConditions
An online audio source that captures the chaos and serendipity of late-night radio tuning in an uncanny audio stream generated on the fly by code.
https://driftconditions.org/

Genres

Ambient, Experimental, Soundscape, Eclectic, Environmental

Stream:

Taglines

- Lost in a sea of midnight static and whispered tales.

- An orchestra of broken dreams and distant echoes.

Description

In the depths of night, an uncanny symphony emerges—a blend of static, whispers, and forgotten melodies. Generated on the fly by code, this stream captures the serendipity of late-night tuning, inviting listeners to explore soundscapes where chance forms haunting harmonies. Each listen is an unexpected journey through a mysterious, ever-evolving tapestry of sound.

Tags

experimental, ambient, soundscape, chaos, serendipity, late-night radio, generative, code-driven, immersive, auditory journey, sonic exploration, unpredictable, atmospheric, haunting, melodic, mysterious, unique listening experience, evolving audio, digital symphony

Inspiration

DriftConditions was 15 years in the making, inspired by an accidental sound collage I heard one night while driving home from my late-night radio show. Between stations, I caught a mix of sermon, newscast, and static. That moment made me wonder: could I recreate this serendipity with code? Now, *DriftConditions* is fully procedurally generated, blending story, music, and sound to capture the unpredictability and magic of that experience, crafted entirely by chance and intention.

Technical Description

The technical foundation of DriftConditions lies in its use of procedural generation to create dynamic audio streams. The system uses JSON5-based recipes to specify the arrangement of audio clips, including classifications, tags, and effects. These recipes act as blueprints, guiding the MixEngine to blend clips into cohesive tracks. The backend manages user contributions and recipe creation, ensuring seamless operation. Dynamic audio effects, such as coherent noise, modulate transitions and amplitudes, creating a harmoniously unpredictable listening experience. This approach mimics the serendipity of late-night radio, offering an ever-evolving soundscape enriched by user-submitted content.

Communication

Member Welcome

Hey {{firstname}},

Welcome to DriftConditions.

You've joined a strange little audio experiment: a continuously running stream built from found sound, field recordings, music, voices, atmospheres, archival fragments, and other beautiful wreckage. It drifts somewhere between late-night radio, dream logic, and the feeling that you've tuned into something you were not quite meant to hear.

I hope you'll check in from time to time for a listen. Or, like some people, leave it on day and night and let it become the weird soundtrack of your life.

And one other thing: if you ever hear this station and think, oh, I have something for this — you probably do.

If you've got a field recording from a lonely roadside, a broken answering machine tape, a church sermon from a thrift-store cassette, distant machinery, shortwave ghosts, archival oddities, a rainstorm, a room tone, a fragment of song, an overheard voice, a strange little sound effect, or some other piece of audio debris that feels like it belongs in this world, you can become a contributor pretty easily.

Just drop me a line and tell me your username and that you want to contribute. I'll upgrade your account, and then you'll be able to upload audio for possible use in the stream.

After that, I'll send along some lightweight guidelines and what kinds of audio tend to work especially well.

So that’s the shape of it: listen, lurk, drift off, come back later, or decide one day that you want to feed the machine.

Glad you found your way here.

Wes

Response to Request for Contribution

Hi {{firstname}},

Thanks for your interest in contributing to DrifftConditions. The station relies on user audio contributions.

Do you have audio you think would fit perfectly within the unique soundscape of the station? We welcome your contributions.

1. Sign up for an account
2. Then let us know you would like to be a contributor. We'll need to know your login.
3. We'll promote you to a contributor
4. After that, you can upload your own audio clips..

After that, you can participate in creating this ever-evolving auditory experience. Once you've signed up, reach out to us to get started. Your input helps shape the dynamic and immersive environment that makes the station so special.

So in short, as a contributor, you can submit whatever you think will add to the vibe and submit as many as you like. But I will also send you some guidelines that might help. A moderator will either approve them or offer feedback.

Promotion to Contributor

Hey, {{firstname}},! You've earned a promotion to contributor!

As a contributor, you can add audio that will be used to make the mix for the station.

To contribute, select Add New Audio from the menu in the top left. You will be invited to upload your audio files.

Here's some tips that will help make sure your contributions are successfully added to the mix:

Title
Use simple meaningful titles that credit the original source. These titles will appear in the playlist.

Classification
Classifications are broad categories that describe the type of audio. **Pro-tip:** Use as few classifications as applicable. Here are the meanings for each classification:

- **Ambient**: Music or sounds that form an ambient bed for other audio, e.g., drone, ambient music, longer static audio, minimalist music
- **Atmospheric**: Includes many field recordings, and longer found sounds, e.g., city ambiance, cafe recordings, crowd noise
- **Environmental**: Recordings of natural sounds, e.g., distant thunder, bird calls, rainfall, ocean waves
- **Premixed**: Already assembled mixes that would not benefit from additional mixing, e.g., radio show/podcast excerpts, mixed music tracks
- **Soundscape**: Complex audio environments combining various sounds, e.g., urban soundscape, virtual environment audio, theatrical sound design
- **Archival**: Historical recordings or sounds, e.g., old radio broadcasts, historical speeches, old TV shows
- **Spoken Word**: Spoken Word recordings, e.g., talks, poetry readings, speeches, interviews
- **Narrative**: Recordings that tell a story, e.g., radio dramas, storytelling sessions, narrative podcasts
- **Instructional**: Educational or instructional audio, e.g., language lessons,

how-to guides, training programs
- **Vocal Music**: Music that prominently features vocals, as opposed to instrumental
- **Instrumental**: Music without vocals, e.g., classical music, instrumental jazz, ambient instrumental tracks
- **Experimental**: Non-traditional or avant-garde audio, e.g., noise music, sound experiments, unconventional audio compositions
- **Digital**: Digital sounds and effects, e.g., synthesizer music, electronic beats, computer-generated sounds
- **Effect**: Short recordings that can be layered over other audio, e.g., sound effects, audio stingers, transition sounds
- **Other**: Miscellaneous sounds that do not fit into other categories, e.g., unique or rare audio clips, uncategorized recordings

Tags

Tags describe the topic, theme, or texture of the audio, such as 'rainy night', 'jazz', or 'whispering'. Use tags to highlight specific elements or moods in your audio. Separate tags with commas. **Pro-tip:** Use as many simple tags as you can think of.

Since tags are used to group similar audio, keep tags simple and plentiful. Rather than compound tags like:

```
1950s-educational-film, propaganda movie, or teenage-angst
```

use simple tags like:

```
1950s, educational, instructional, propaganda, teenage, teenager, angst
```

Don't worry about providing all possible plural tags as the system will already try to work that out automatically.

A Word About Copyright

In general, we think copyright is too often a way of locking in the profits of

corporations, however we do want to show respect for other artists. So far, I've used only stuff that was either:

- in the public domain, e.g., stuff on archive.org or published before 1924
- creative commons, e.g., material on freemusicarchive.org or freemusicarchive.org
- public performance captured by a spectator, e.g., street performers, gamelan beleganjur
- so old and influential, in our opinion, it could be said to be owned by everyone, e.g., Jelly Roll Morton, Texas Swing, Ink Spots
- so obscure, the crime of stealing it is offset by the opportunity that someone actually gets to hear it, e.g., Tuvan throat singers, Baka water music

Hopefully, this will be helpful (and not overwhelming) information that will guide you as you contribute.

Promotion to Editor

Hey {{firstname}},

The material you've been submitting has been spot-on. So you've earned a promotion to editor! You'll see a few more actions on the menu:

- List All Audio
- Edit Audio
- Submit Batch Audio

You'll also notice that you can self-approve your submissions, even when you submit them.

Use your new powers wisely and carefully!

If you have questions, don't be afraid to ask.

Posted to Social Media

a stream built from an accident I heard between two radio stations

Late one night in the early oughts, I was driving home from my radio show on KUSP Santa Cruz in my old pickup truck — a late-night aural collage of music, story, and found sound — when I stumbled across something strange on the dial. Someone was layering a sermon with BBC news, lacing it with static, the whole thing drifting in and out. I pulled over on a dark mountain highway to listen. I eventually lost the signal. When I tuned back in, I realized I'd been parked between two stations the whole time, and the gorgeous mess had been an accident. That's when I asked: if something this beautiful can happen by accident, can you make it happen on purpose?

15 years later, I created DriftConditions. It's a 24/7 stream that never repeats. Code pulls from a library of contributed audio — ambient beds, spoken word, weird interjections — and weaves them together with dynamic modulation that gives it the feel of radio interference. JSON recipes tell the MixEngine what to grab and how to stack it.

The numbers, since this is Reddit and you'll ask: 28 recipes pulling from 1,649 audio clips, generating 140,366 unique mixes across 16,872 hours of uptime. On top of that, the recent push: 79 additions, changes, and fixes across 42,000 lines of code. Stream is at driftconditions.org.

The library is contributed by a small handful of people. Always needs more strange and beautiful things. Field recordings are the backbone — the long atmospheric stuff is what voices drift across.

If you've got audio sitting around, there's a contributor path. If not, put it on at night with headphones. Give it ten minutes.

Happy to nerd out on the stack if anyone wants — I'll spare you unless asked.

Here's a little sample:

[video sample with generative audio]

crafting recipes for a procedural audio stream — how loose is loose enough?

How do you tune the looseness in a generative system? How do you know when you've gotten too specific and killed the surprise, or too loose and lost the idea entirely? Do you tune by ear, by tweaking parameters, by gut?

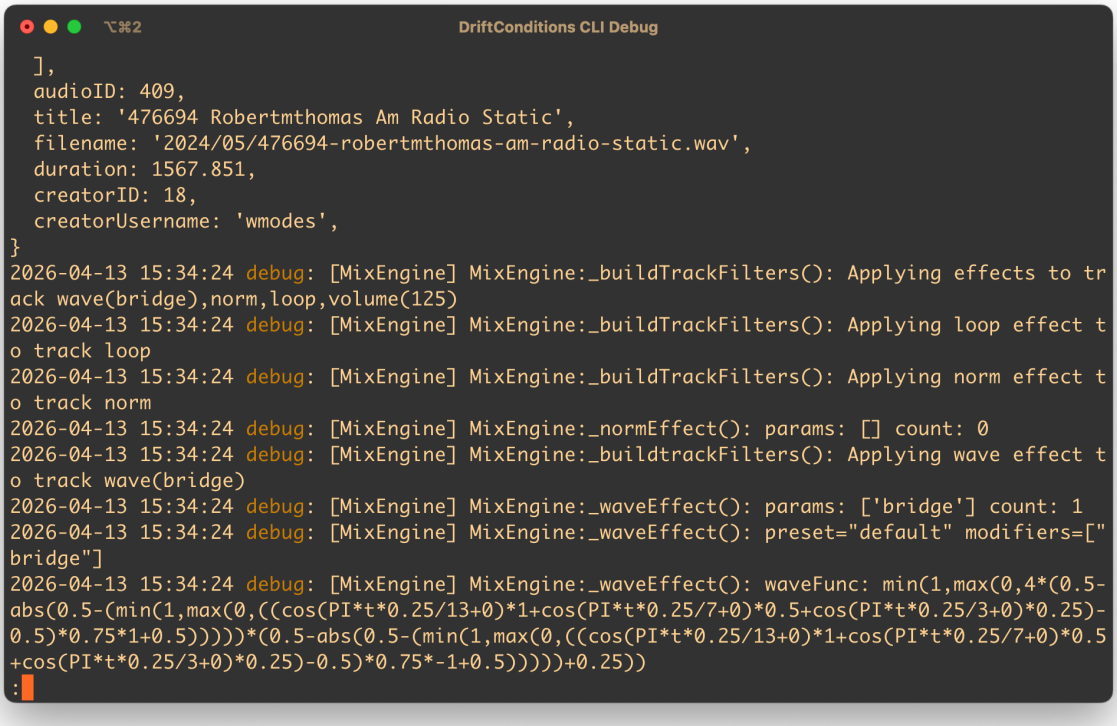
I created a procedurally generated audio stream called DriftConditions (driftconditions.org). Every mix in the stream is based on a human-made recipe. Creators start with a general idea of what they want to hear — a theory of the case. Something like "Long Narrative with Music Bed," "Interrupted Sermon," or "Fucked Up Radio Aircheck." These are all real recipes on DriftConditions. Then the creator attempts to craft a recipe that has enough specificity to capture their idea, but enough looseness to allow for happy accidents.

Recipe Data: *

```
{
  tracks: [
    {
      // this is the sermon that fades in and out
      label: "sermon",
      volume: 100,
      effects: ["wave()"],
      clips: [
        {
          classification: ["Spoken"],
          tags: ["sermon", "religion"],
          length: ["medium", "long", "huge"]
        }
      ]
    },
    {
      // this is the newscast that fades out and in
      label: "newscast",
      volume: 100,
      effects: ["wave(counter)"],
      clips: [
        {
          classification: ["Archival", "Spoken"],
          tags: ["news", "weather", "traffic", "radio", "aircheck"],
          length: ["medium", "long", "huge"]
        },
        {
          classification: ["Archival", "Spoken"],
          tags: ["news", "weather", "traffic", "radio", "aircheck"],
          length: ["medium", "long", "huge"]
        },
        {
```

Reset | Validate | Add Track | Insert Clip | Insert Silence

Recipe for Interrupted Sermon



Recipe being transformed into a mix

When I'm crafting recipes, one thing that serves as inspiration is to listen to experimental audio. If I try to capture the spirit of something I particularly like, the gulf between my intention and what the system creates is where the magic lies.

So back to the question: what's your process?

Here's a sample:

[video sample with generative audio]